

Design a Chat System

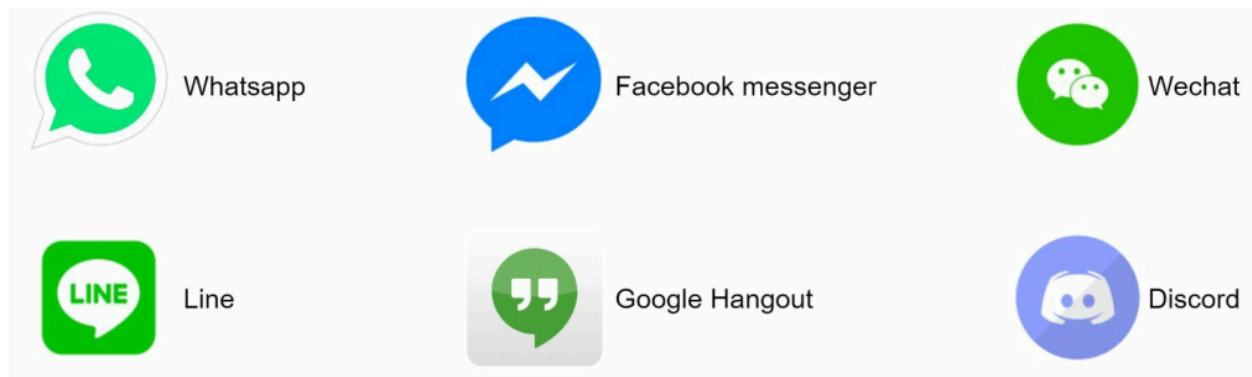
1. Introduction

A **Chat System** enables users to communicate in real time through:

- One-to-one messaging
- Group chats
- Media sharing
- Online/offline messaging

Popular chat applications include:

- WhatsApp
- Facebook Messenger
- Telegram
- WeChat
- Slack



2. Importance of Requirement Clarification

Chat systems can vary greatly depending on:

- One-to-one chat vs group chat

- Small-scale vs large-scale system
- Real-time vs asynchronous communication
- Media support
- Message persistence

Therefore, the first step is to clearly understand the interviewer's expectations before designing the architecture.

3. Common Features of a Chat System

One-to-One Chat

- Direct communication between two users
- Low latency messaging
- Message delivery confirmation

Group Chat

- Multiple users communicate in the same chat room
- Requires message synchronization among participants

Media Sharing

Supports sending:

- Images
- Videos
- Documents
- Audio files

Online Presence

Shows user status such as:

- Online
- Offline
- Last seen

Push Notifications

Users receive alerts when:

- New messages arrive
- They are mentioned
- Group activity occurs

Message Synchronization

Messages should sync across:

- Mobile devices
- Tablets
- Web browsers
- Desktop apps

4. Core Challenges in Chat System Design

Low Latency

Messages should be delivered almost instantly.

Scalability

The system must support:

- Millions of concurrent users
- High message throughput

Reliability

Messages should:

- Not be lost
- Be delivered correctly
- Be stored persistently

Distributed System Challenges

Need to handle:

- Multiple servers
- Network failures
- Regional traffic

Security

Chat systems require:

- Authentication
- Authorization
- Encryption

5. Key Components in Chat Architecture

Component	Purpose
-----------	---------

Chat Client	Sends/receives messages
API Gateway	Handles requests
Chat Server	Core messaging logic
Presence Server	Tracks online users
Notification Service	Push notifications
Message Queue	Async processing
Database	Stores chat history
Cache	Fast message retrieval

6. Communication Protocols

Common protocols used:

Protocol	Usage
HTTP	Login, APIs

WebSocket	Real-time communication
TCP	Persistent connection
Push Services	Mobile notifications

7. High-Level Goals

A good chat system should provide:

- Real-time messaging
- High availability
- Horizontal scalability
- Reliable delivery
- Multi-device synchronization
- Efficient storage

Key Takeaway

Chat systems are fundamentally real-time distributed systems.

The biggest challenges are:

- Maintaining persistent connections
- Delivering messages quickly
- Synchronizing messages across devices
- Scaling to millions of users

Requirement clarification is extremely important because system architecture changes significantly depending on product requirements.